

DATA 298B MSDA Project II

Workbook 1

WildTrace: From Wildlife Photos to Robot Pen Drawings — Line-Drawing Dataset Pipeline and xArm7 Drawing in Isaac Sim

Team: ____

Members:

[Member 1 full name]

[Member 2 full name]

[Member 3 full name]

[Member 4 full name]

[Member 5 full name]

Date: 10/04/2026

GitHub Link:

<https://github.com/shra012/WildTrace> (branch: development)

4. Model Development

Robot drawing and imitation-learning projects need many clean, varied line drawings that a robot arm can actually execute. Hand-made datasets are small and synthetic ones look nothing like real animals. WildTrace builds that data automatically from real wildlife photographs and then proves each result on a simulated robot. The system has two layers:

1. **The Dataset Layer (WildTrace pipeline)** turns Open Images v7 photos into validated line drawings and gold pen trajectories. It runs as a bronze → silver → gold medallion pipeline:
 - Curates subjects using segmentation masks, image quality checks, a viewpoint classifier and BioCLIP labels
 - Redraws each subject as a clean outline with a generative image model (FLUX.1-schnell)
 - Validates each drawing with an OpenCV pre-screen and a vision-language-model (VLM) judge inside an agentic LangGraph retry loop, then exports at most 6 strokes per drawing as normalized pen trajectories
2. **The Execution Layer (Isaac Sim xArm7)** draws a gold trajectory with a simulated UFACTORY xArm7 holding a pen and measures how closely the pen tip followed the intended path:
 - Maps the drawing onto a 16 × 12 cm paper area and shapes the path (corner-aware smoothing, resampling)
 - Solves inverse kinematics every physics tick with NVIDIA's Lula solver inside a waypoint state machine
 - Writes tracking-error metrics (RMSE, max error, draw time) stamped with the git SHA and config hash of the run, so every number is reproducible

A model-benchmark harness compares candidate generators and judges on fixed evaluation sets, and two Model Context Protocol (MCP) servers let an operator or an AI agent run the whole loop ("draw a frog") from one command.

4.1 Model Proposals

1. Subject Curation and Enrichment (BioCLIP + Viewpoint Classifier)

Not every photo can become a good drawing. The animal must be whole, in focus, large enough and seen from an angle a simple outline can express. This stage filters raw photos down to usable subjects and labels each one with its species and viewing angle.

Dataset and Preprocessing

We use the **Open Images v7** train split: images plus human-annotated instance segmentation masks, all CC BY 2.0 licensed. One sample is one mask instance. Seven categories are configured (Cat, Dog, Horse, Bird, Butterfly and Fish at 100 each, Frog at 40). Butterfly has no train masks in Open Images, so it yields no samples. The fetch ledger holds 646 rows (641 succeeded, 5 failed on corrupt mask archives), and the latest view holds **540 samples**.

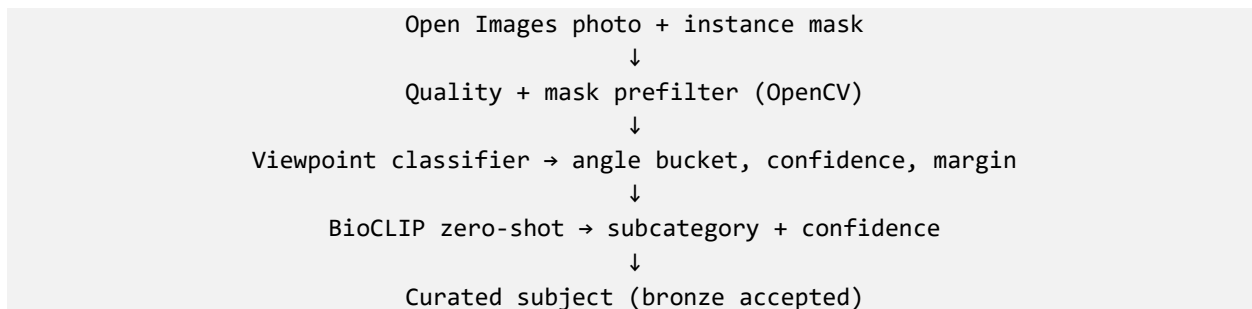
Curation applies:

- Duplicate removal by image checksum
- Minimum size 256×256 px, resize to at most 1024 px, blur variance ≥ 0.001
- Mask prefilter: coverage ≥ 0.03 , border touch ≤ 0.50 , ≤ 4 components, largest component ≥ 0.75 , bounding-box fill ≥ 0.10 , top-mass dominance ≤ 0.18
- Viewpoint gate: only left/right profile, front and front three-quarter views are kept; rear, top-down and occluded views are rejected (min confidence 0.42, min margin 0.08)

Result: **186 of 540 samples accepted (34.4%)**. The main rejection reasons were low mask coverage (169), top-heavy pose (141) and blur (102).

Model Architecture

- **BioCLIP** (imageomics/bioclclip), a CLIP-style vision-language model trained for biology, scores each crop against candidate species prompts ("a photo of a {label}"); the top label becomes the subcategory. Mean top-1 confidence is 0.885 on accepted subjects (Cat 0.771, Dog 0.810, Bird 0.904).
- **Viewpoint classifier (local_score_v1)** is a transparent geometric model on the mask: flip-loU symmetry, principal-axis angle, left/right/top/bottom mass and a facing-direction score feed weighted scores for each viewpoint bucket. Confidence = top score $\div$ sum; margin = top – second. Mean view confidence is 0.674.



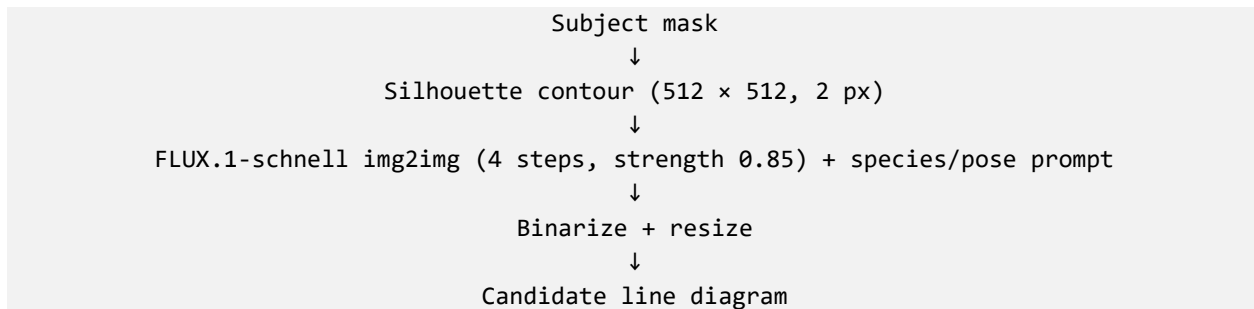
2. Outline Rectifier (FLUX.1-schnell Image-to-Image)

The rectifier turns a curated subject into a simple black-on-white outline drawing. Classical edge detection keeps fur, background and texture; a generative model can redraw the animal as a clean coloring-book outline.

Model Architecture and Configuration

- Stage 1, silhouette contour: the subject mask is resized to 512×512 , closed with a 7×7 ellipse (3 iterations), and its external contours (area > 200 px) are simplified and drawn as 2 px lines.
- Stage 2, FLUX.1-schnell image-to-image: a 12B-parameter rectified-flow transformer, run as a 4-bit GGUF quantization (city96/FLUX.1-schnell-gguf, Q4_K_S) with bfloat16 compute.
- Settings: 4 inference steps, guidance 0.0, strength 0.85, conditioning on the isolated subject.
- The prompt is built from species, breed, a pose estimate and the viewing angle, for example "simple clean line drawing of a cat, standing, side left, coloring book page for children, black outline on white background, minimalist doodle, no shading, no fill, no color".
- The output is binarized (threshold 128) and resized back to the crop size.

Runtime: the 186 first-attempt generations took 1.29 hours (median 24.2 s per sample) on the laptop GPU with model CPU offload, which uses about 20 GB of host RAM.

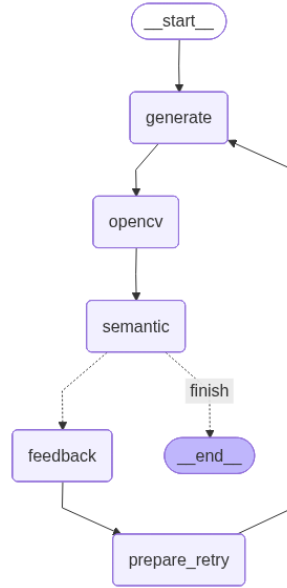


3. Diagram Validator (OpenCV Pre-screen + VLM Judge in a LangGraph Agent)

Generated drawings fail in predictable ways: blobs, fragmented strokes, missing legs or filled silhouettes. The validator catches these before a drawing reaches the robot, and an agent retries generation with adjusted settings.

Model Architecture

- **OpenCV pre-screen:** ink ratio 0.01–0.32, ≤ 18 connected components, small-contour ratio ≤ 0.70 , body bottom reach ≥ 0.60 , ≤ 5 fragments in the bottom 35%. Score = $1 - 0.8 \cdot |\text{ink} - 0.16| - 0.03 \cdot \max(\text{components} - 3, 0) - 0.6 \cdot \text{small} - 0.5 \cdot \max(0, 0.60 - \text{reach}) - 0.04 \cdot \max(\text{fragments} - 3, 0)$.
- **VLM judge:** qwen2.5vl:7b served locally by Ollama (temperature 0). It returns JSON {passed, score, reason}; a drawing passes when the verdict is true and the score is ≥ 0.55 . Claude Haiku 4.5 and OpenRouter-hosted models are configurable alternatives.
- **Combined score** = $0.5 \times \text{OpenCV score} + 0.5 \times \text{VLM score}$.
- **LangGraph retry agent:** generate $\rightarrow$ OpenCV $\rightarrow$ semantic judge $\rightarrow$ finish, or feedback $\rightarrow$ prepare retry $\rightarrow$ generate. Strength moves by ± 0.05 depending on the failure (clamped 0.60–0.98), and the VLM can suggest new parameters. At most 5 attempts.



LangGraph Validation Agent

Results on the 186 subjects: **132 diagrams accepted (71.0%)**, with 51.1% passing on the first attempt (attempts for accepted drawings: 1 → 95, 2 → 20, 3 → 7, 4 → 5, 5 → 5). All 54 rejected subjects used all 5 attempts and failed the OpenCV pre-screen (too many components 49, too many small contours 47, too many lower-body fragments 34). Mean combined score of accepted drawings is 0.632.

4. Trajectory Extraction and Gold Export

A drawing becomes robot input when it is a short list of pen strokes. For each (category, subcategory, angle) bucket the highest-scoring accepted drawing is selected, then its outline contours are traced:

- Threshold < 180, outer contours only, longest first, top 6 kept, strokes under 16 points dropped
- Each stroke subsampled to ≤ 128 points and normalized to a 0–1 canvas; exported as JSON, SVG and an NDJSON gold record with lineage and QA scores

Result: **39 gold trajectories** across 6 categories, covering 26 of 30 category × angle cells. Mean 4.46 strokes (range 2–6) and 379.5 points (range 155–768) per drawing; 16 of 39 hit the 6-stroke cap.

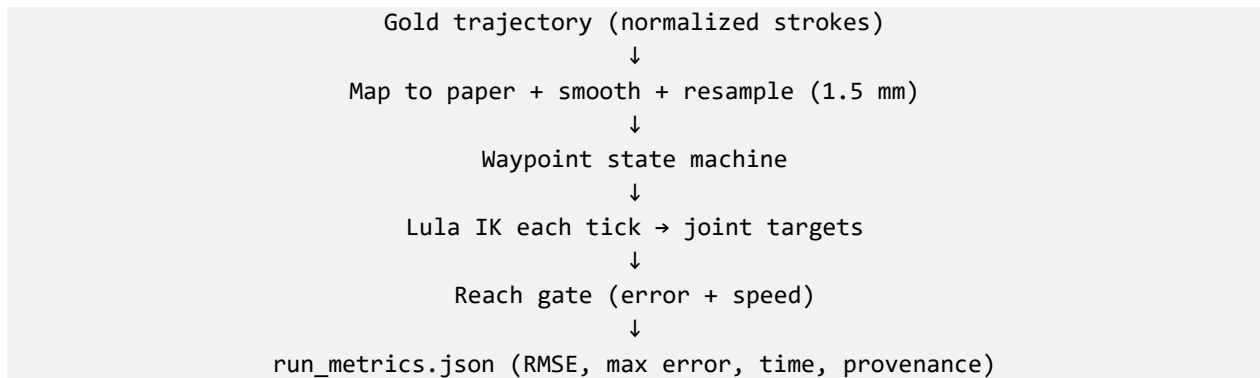
5. Robot Drawing Controller (xArm7 + Lula IK in Isaac Sim)

The controller executes a gold trajectory on a simulated UFACTORY xArm7 (official xarm_ros2 description) with a 120 mm pen whose tip frame sits 0.12 m beyond the end effector.

Model Architecture

- **Coordinate mapping:** uniform scale onto a 16 × 12 cm area centred at (0.45, 0) m, Y flipped, Laplacian smoothing (strength 0.40) that preserves corners sharper than 28°, resampling to ≤ 1.5 mm steps.

- **State machine:** HOME → APPROACH → LOWER PEN → DRAW → LIFT PEN → MOVE TO NEXT STROKE → ... → FINISHED. It advances only when a waypoint is reached; a timeout fails safely.
- **Inverse kinematics:** NVIDIA Lula kinematics solver, solved every 1/60 s tick. Solutions that are non-finite or outside joint limits are rejected.
- **Reach gate:** position error ≤ 1.2 mm (0.6 mm at corners) AND pen-tip speed ≤ 7 mm/s before advancing. Joint drives run at kp 400, kd 95.
- **Pen heights:** down 0.201 m (paper at 0.200 m), up 0.235 m, approach 0.27 m.



6. Behavior Cloning Policy (prepared, future work)

To learn drawing from demonstrations instead of solving IK, a behavior cloning policy is implemented: a multilayer perceptron [256, 256, 128] with ReLU, 36 input features and a tanh-bounded Cartesian delta output of at most 5 mm per step (Adam 1e-3, MSE, batch 128, early stopping). One demonstration exists (8,760 frames of a two-stroke puppy); training is deliberately blocked until at least two drawing IDs exist, so the policy has not been trained yet. The 39 gold trajectories are the planned source of demonstrations.

4.2 Model Supports

4.2.1 Development and Execution Environment

All models run on one laptop so results stay easy to reproduce and monitor.

Development Stack

- Python 3.12, PyTorch 2.11 (CUDA 13.0), Hugging Face diffusers 0.37 and transformers 5.5
- open-clip (BioCLIP), OpenCV 4.13, LangGraph 1.1, ONNX Runtime 1.30
- Ollama for local VLMs; OpenRouter for hosted VLM judges in benchmarks
- NVIDIA Isaac Sim 6.0.1 with PhysX and the Lula kinematics solver
- uv for dependency locking; Git and GitHub for versioning and issues
- Hardware: NVIDIA RTX 3080 Ti Laptop GPU (16 GB VRAM), Intel i9-12900HK, 23 GB RAM + 6 GB swap available to WSL2 on Windows 11

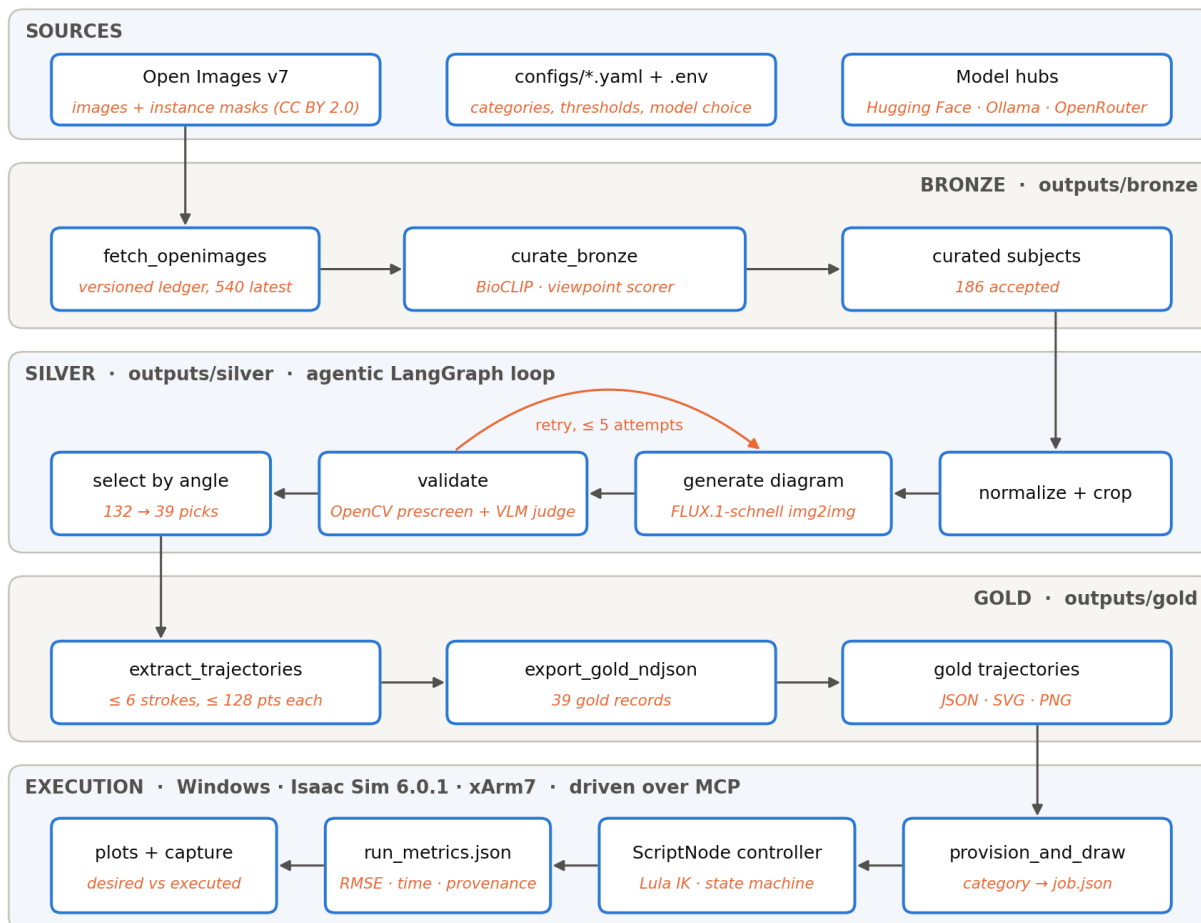
Every model follows the same systematic pipeline:

- Configuration from YAML + .env (thresholds, model names, seeds)
- Idempotent stage script that reads the previous stage's NDJSON manifest

- Per-record lineage and QA scores written to the next manifest
- Evaluation against fixed evaluation sets with bootstrap confidence intervals
- Provenance stamping (git SHA + config hash) on every robot run
- Results tables written to Markdown and Word reports

4.2.2 System Architecture Overview

The system follows a layered, multi-stage architecture: data curation before generation, validation after generation, and physical (simulated) verification at the end.



End-to-End System Architecture

Dataset Layer: generation and validation

1. Fetch and curation reduce raw Open Images samples to subjects with clean masks and usable angles.
2. The rectifier generates a candidate drawing from the subject's silhouette.
3. The validator scores it with OpenCV and the VLM judge and routes it (table below).

- Selection keeps the best drawing per category × subcategory × angle bucket, and export writes gold trajectories.

Validation outcome	Action
OpenCV pre-screen fails	Skip the VLM judge; adjust strength; retry
OpenCV passes, VLM verdict false or score < 0.55	Adjust strength (VLM may suggest values); retry
OpenCV passes, VLM passes with score ≥ 0.55	Accept; record combined score
5 attempts used without passing	Reject; record failure flags

Execution Layer: simulated drawing

- provision_and_draw picks an undrawn gold trajectory for a requested category, or reports not_available with no side effects.
- It writes job.json; the Isaac Sim ScriptNode controller reads it at the start of the next run.
- The controller draws the trajectory and writes run_metrics.json, a comparison plot and a viewport capture.

4.3 Model Comparison and Justification

4.3.1 Comparison Across Model Categories

1. Master Model Comparison: All Components

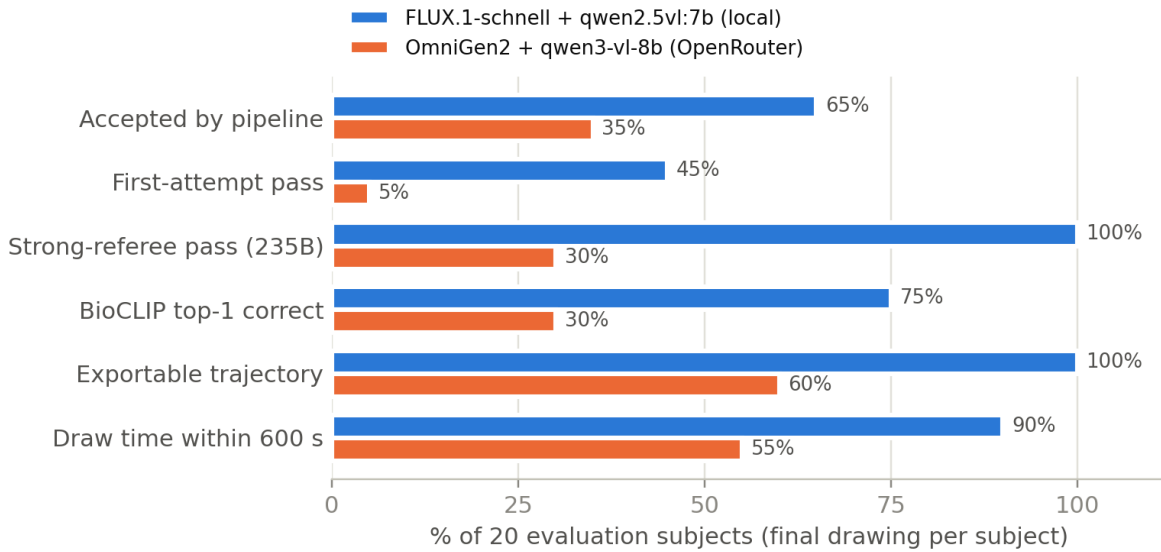
Scores come from benchmark r2 (20 subjects per setup) and the baseline-v1 robot runs. Rows marked "not yet benchmarked" are implemented in the code but have not been measured.

Component	Model Evaluated	Model Type	Key Metric	Score	Select ed
Outline Rectifier	★ FLUX.1-schnell (GGUF Q4)	Rectified-flow img2img	Strong-referee pass	100%	✓
	OmniGen2 (Qwen2.5-VL-3B + 4B decoder)	Unified VLM + diffusion edit	Strong-referee pass	30%	
	FLUX.1-Kontext-dev, SD1.5 + ControlNet lineart, informative-drawings	Edit / ControlNet / GAN	—	Not yet benchmark ed	
Diagram Judge	★ qwen3-vl-8b-instruct	VLM	Cohen's κ vs strong referee	0.83	✓
	qwen2.5vl:7b (current production)	VLM	Cohen's κ vs strong referee	0.00	
	qwen3-vl-235b-a22b (strong referee)	MoE VLM	Reference judge	—	Ref.
Robot Controller	★ baseline-v1 (Lula IK, velocity gate)	IK + state machine	Tracking RMSE (mm)	0.336	✓
	Pre-baseline controller (c3408fc)	IK + state machine	Tracking RMSE (mm)	1.450	
Subject Labels	★ BioCLIP	CLIP-style VLM	Mean top-1 confidence	0.885	✓

2. Generator + Judge Setups (benchmark r2)

Each setup ran the production LangGraph loop (max 3 attempts) on the same 20 subjects. A 235B referee then re-judged every final drawing.

Metric	★ FLUX.1-schnell + qwen2.5vl:7b (local)	OmniGen2 + qwen3-vl-8b (OpenRouter)
Accepted / first-attempt pass	65% / 45%	35% / 5%
Mean attempts / combined score	2.00 / 0.43	2.75 / 0.42
Strong-referee (235B) pass / mean score	100% / 0.89	30% / 0.31
BioCLIP top-1 correct / p(true)	75% / 0.73	30% / 0.24
Silhouette IoU with source mask	0.20	0.28
Blank final drawings	0%	45%
Exportable trajectory / ink coverage	100% / 0.63	60% / 0.23
Estimated draw time / within 600 s	428 s / 90%	414 s / 55%
Time per subject / generation p50	113 s / 45.8 s	202 s / 71.3 s
Peak VRAM / system RAM	11.3 GB / 22.9 GB	9.0 GB / 20.9 GB
API cost for the run	\$0	\$0.0008



Benchmark r2: Generator + Judge Setups (n = 20 each)

3. Judge Agreement (40 final drawings, 3 referees)

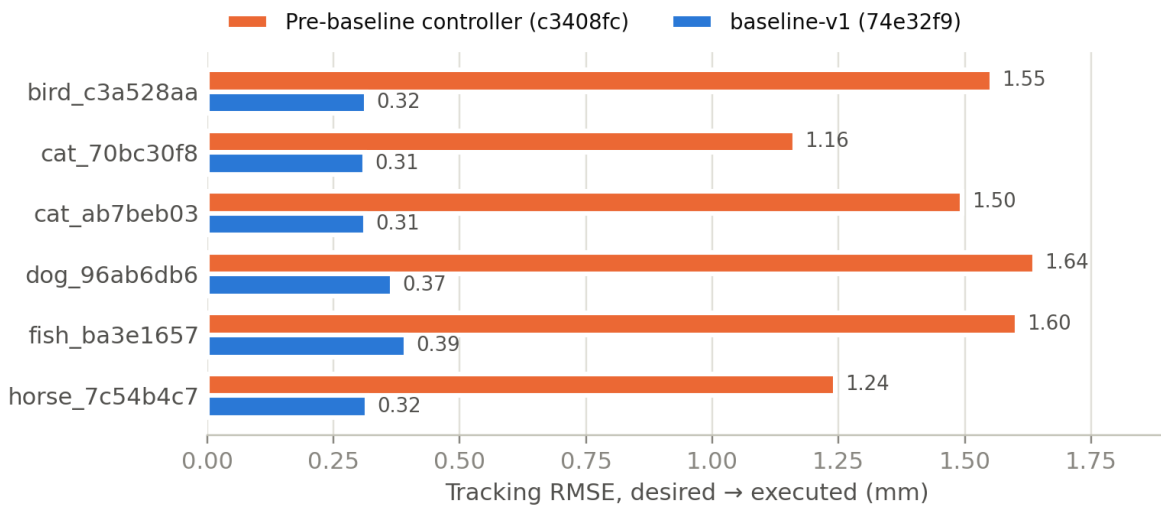
Judge pair	Agreement	Cohen's κ	Notes
qwen3-vl-8b vs qwen3-vl-235b	92.5%	0.827	8B matches the strong referee closely
qwen2.5vl:7b vs qwen3-vl-235b	65.0%	0.00	7B passed 40/40, including 9 blank drawings
qwen3-vl-8b vs qwen2.5vl:7b	72.5%	0.00	Production judge does not discriminate

Referee latency (median): qwen3-vl-8b 1.16 s, qwen2.5vl:7b 1.8 s (warm), qwen3-vl-235b 3.35 s. Referee cost per 40 calls: \$0.0045 for the 8B and \$0.009 for the 235B model.

4. Robot Controller Tuning (6 paired gold trajectories)

Metric	Pre-baseline (c3408fc)	★ baseline-v1 (74e32f9)	Change
Tracking RMSE, desired → executed	1.450 mm	0.336 mm	4.3× lower
Max error, desired → executed	2.927 mm	1.090 mm	2.7× lower
RMSE, executed → desired	3.270 mm	0.728 mm	4.5× lower
Waypoint reach RMSE	2.813 mm	0.954 mm	2.9× lower
Mean draw time (simulated)	191.6 s	369.2 s	1.93× slower

The tuned controller adds a 7 mm/s velocity gate, a tighter 1.2 mm reach gate (0.6 mm at corners), smaller 1.5 mm steps, stronger damping (kd 40 → 95) and densified corner waypoints. It is slower, but the longest drawing (horse, 8 strokes) still finishes in 562.9 s, inside the 10-minute budget planned for the real robot.



Controller Tuning: Tracking RMSE Before and After

Key Observations

1. FLUX.1-schnell beat OmniGen2 on every robot-relevant measure: strong-referee pass 20/20 vs 6/20 (sign test $p = 0.0001$), BioCLIP recognizability 75% vs 30% ($p = 0.0004$) and exportable trajectories 100% vs 60%, at 1.8× lower time per subject.
2. OmniGen2 follows the source pose more faithfully when it draws (IoU 0.28 vs 0.20, not significant at $n = 20$), but 45% of its final drawings were blank and many were filled silhouettes.
3. The production judge qwen2.5vl:7b does not discriminate: it passed every drawing, blanks included, so in practice only the OpenCV pre-screen rejects anything. qwen3-vl-8b agrees with the 235B referee (κ 0.83) at about one second per call.
4. Controller tuning cut tracking error 4.3× at the cost of 1.9× longer draws, a trade-off issue #16 will tune further.

4.3.2 Generation Strategies Compared

We compared two complete generation strategies and selected a hybrid of their strengths.

Strategy 1: Contour-Conditioned Diffusion (FLUX.1-schnell + local judge)

Advantages:

- Always produces clean, connected outlines (0% blank, 100% exportable)
- Drawings are recognizable to an independent model (BioCLIP top-1 75%)
- Fully local, no API cost; fastest per subject (113 s)

Disadvantages:

- Draws stock poses; a lying cat becomes a sitting cat (median IoU ≈ 0.2)
- Needs ~20 GB host RAM with CPU offload; cannot share memory with the judge

Strategy 2: Photo-Conditioned Unified Editing (OmniGen2 + hosted judge)

Advantages:

- Reads the masked photo, so successful drawings keep the source pose (IoU up to 0.78–0.91)
- Hosted judge (qwen3-vl-8b) is fast (~1 s) and discriminating

Disadvantages:

- 45% blank final drawings; a fixed seed stayed blank on every retry until seeds were varied per attempt
- Thin, broken strokes fail the component gates; only 60% produce a usable trajectory

Strategy 3: Final Choice (FLUX.1-schnell + qwen3-vl-8b judge)

Keep FLUX.1-schnell as the generator and replace the production judge with qwen3-vl-8b, which matches the strong referee. Hence the selected configuration is:

```
Silhouette contour → FLUX.1-schnell → OpenCV pre-screen → qwen3-vl-8b judge → gold trajectory
```

4.4 Model Evaluation Methods

4.4.1 Evaluation Design

Evaluation sets

- **evalset_v2** (benchmark r2): 20 subjects, 3 per category plus 1 extra Cat and Dog, seed 14
- **evalset_v1**: 30 subjects, 5 per category, seed 14 (used by the per-model harness)
- **labelset_v1**: 60 diagrams (44 pipeline-accepted, 16 rejected) for human accept/reject labels. Labels have not been collected yet, so the 235B referee serves as a proxy, not as ground truth.

Metric groups (wildtrace/eval_metrics.py)

Group	What it measures	Metrics
A. Pipeline outcome	Did the pipeline accept the drawing?	Acceptance, first-attempt pass, attempts, combined score, error rate
B. Drawing structure	Is it a clean line drawing?	OpenCV pass/score, ink ratio, components, small-contour ratio, bottom reach, fill ratio, stroke width, blank
C. Shape fidelity	Does it match the source animal's shape?	Silhouette IoU, Chamfer distance (% diagonal), boundary F-score, aspect error, centroid offset
D. Pose	Same angle and facing direction?	Angle-bucket match, direction-sign match, $ \Delta \text{ direction} $, $ \Delta \text{ symmetry} $
E. Recognizability	Can an independent model name the animal?	BioCLIP top-1, p(true class), drawing-to-photo cosine
F. Judges	Do judges agree?	Own-judge pass, parse failures, latency, cross-referee pass, Cohen's κ
G. Robot drawability	Can the robot draw it in time?	Strokes (1–6), exportable, coverage, pen-down/up mm, estimated draw time, within 600 s
H. Cost and resources	What does it cost to run?	Generation latency, time per subject, peak VRAM/RAM, API cost

Statistics

- Bootstrap 95% confidence intervals (2,000 resamples, seed 14) for every rate and mean
- Exact two-sided sign test on paired per-subject differences (ties dropped)
- Cohen's κ for judge agreement
- Draw-time model fitted on 7 finished Isaac Sim runs: $t = 0.3528 \cdot \text{pen mm} + 26.28 \cdot \text{strokes} + 4.52 \text{ s}$ ($R^2 = 0.989$)

Robot tracking metrics (src/metrics.py)

- **Desired** → **executed RMSE** (headline): for every desired pen-down point, the distance to the nearest executed pen-tip sample. It measures how much of the intended drawing was missed.
- **Executed** → **desired RMSE**: the reverse direction; it measures stray ink.
- **Waypoint reach error**: the error at the moment each waypoint was accepted (bounded by the reach gate).
- Each run also records max, mean and p95 error, simulated duration, visited states, git SHA and config hash.

4.4.2 Model Validation and Evaluation Results

Paired per-subject comparison of the two r2 setups (wins for FLUX / OmniGen2 / ties):

Measure	FLUX wins	OmniGen2 wins	Ties	Sign-test p
Strong-referee score	18	1	1	0.0001
BioCLIP p(true class)	18	2	0	0.0004
Ink coverage by trajectory	19	1	0	< 0.0001
Silhouette IoU	12	8	0	0.50
Chamfer distance	12	8	0	0.50

Per-category view (FLUX / OmniGen2):

Category	Accepted	Strong-referee pass	BioCLIP top-1	IoU
Bird	33% / 33%	100% / 33%	100% / 33%	0.11 / 0.28
Cat	50% / 50%	100% / 50%	50% / 0%	0.26 / 0.24
Dog	50% / 50%	100% / 25%	100% / 75%	0.17 / 0.17
Fish	100% / 0%	100% / 33%	0% / 0%	0.21 / 0.55
Frog	100% / 0%	100% / 0%	100% / 33%	0.20 / 0.30
Horse	67% / 67%	100% / 33%	100% / 33%	0.20 / 0.21



Validation with Samples: Source Photo, OmniGen2 and FLUX Drawings (r2)

4.4.3 Robot Drawing Evaluation (baseline-v1)

Every gold trajectory available at the time was drawn under the frozen baseline-v1 configuration (git tag baseline-v1, config hash 15e6428dffca). All runs finished.

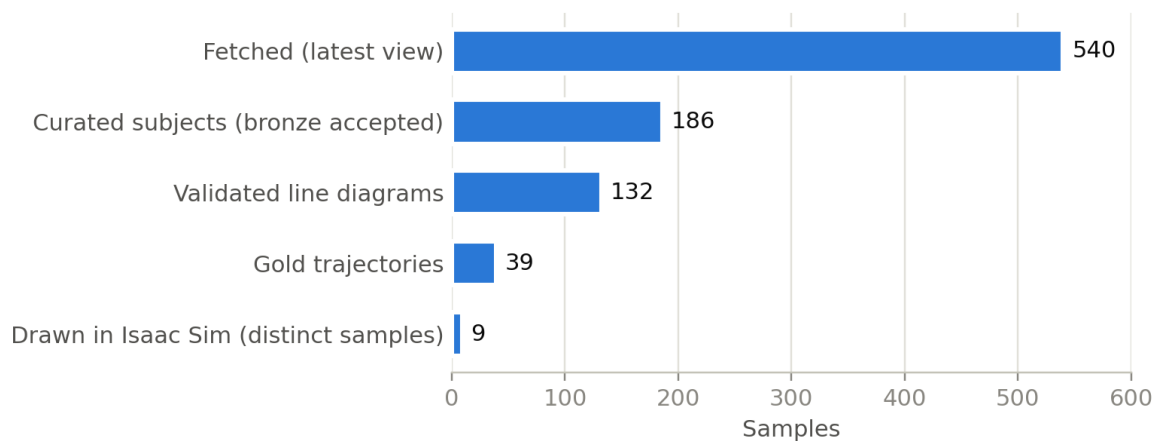
Trajectory	Strokes	Pen-down pts	RMSE (mm)	Max (mm)	Draw time (s)
bird_c3a528aa	4	558	0.315	1.023	312.6
cat_70bc30f8	5	910	0.312	1.127	425.1
cat_ab7beb03	4	388	0.314	0.967	241.1
cat_4651e893 (stress)	6	954	0.270	1.025	427.9
dog_96ab6db6	4	648	0.366	1.084	380.4
fish_ba3e1657	3	656	0.394	1.174	293.4
frog_a9263f39	6	905	0.355	1.019	529.9
horse_7c54b4c7	8	902	0.316	1.167	562.9
frog_3f221bad (new, reproduction check)	2	533	0.366	1.024	277.1

Frozen set (first 8): **mean RMSE 0.330 mm** (range 0.270–0.394 mm), max error 0.97–1.17 mm, 8/8 finished. The later frog_3f221bad run landed inside that range (0.366 mm), a first check that the baseline reproduces.

Context from published robot pen-writing results

Source	System / task	Reported error
arXiv 2609.11775	In-hand pen writing (real hardware)	0.64 ± 0.10 mm
arXiv 2609.11775	3-finger gripper, free-space waypoints	0.41–0.45 mm
arXiv 2609.11775	Shadow Hand, in-hand writing	1.48 mm RMSE
arXiv 2407.19826	Six-DOF hybrid arm, general tracking	≈ 0.38 mm RMSE
WildTrace baseline-v1	xArm7 pen drawing, simulation	0.270–0.394 mm RMSE

Caveat: our numbers come from a zero-gravity, contact-free simulation, while the published numbers come from real hardware. The comparison shows the controller is in the right range; it does not show that it beats those systems.



Samples Surviving Each Stage, Photo to Robot

5. Data Analytics System

5.1 System Requirements Analysis

5.1.1 System Boundary, Actors, and Use Cases

System Boundary

The system is a research-grade dataset and verification framework that turns public wildlife photos into robot-ready drawing trajectories and verifies them on a simulated robot arm.

Inside System Boundary

1. Open Images fetch with a versioned, idempotent ledger
2. Curation: quality filters, viewpoint classifier, BioCLIP enrichment
3. Silver normalization, subject isolation and cropping
4. Outline rectifiers (FLUX.1-schnell and alternatives)
5. Validation: OpenCV pre-screen, VLM judge, LangGraph retry agent
6. Angle-bucket selection, trajectory extraction and gold export
7. Isaac Sim scene build, xArm7 drawing controller and tracking metrics
8. MCP orchestration server (wildtrace-sim) and the draw-animal workflow
9. Benchmark harness, evaluation metrics and reports
10. Provenance stamping and the frozen baseline configuration

Outside System Boundary

- Open Images v7 dataset hosting (images, masks, annotation CSVs)
- Model hubs and runtimes: Hugging Face Hub, Ollama, OpenRouter API
- NVIDIA Isaac Sim application and the third-party isaacsim-mcp-server on Windows
- GitHub (code hosting, issues, release tags)
- A physical xArm7 robot (future integration; no network client exists today)

Actors

Actor	Role
Data / ML Engineer	Runs pipeline stages, tunes thresholds, adds categories, swaps models
Robotics Engineer	Tunes the controller, freezes baselines, reviews tracking metrics
Reviewer / Annotator	Labels drawings accept/reject (planned labeling set of 60)
AI Agent (MCP client)	Starts Isaac Sim, builds the scene and runs "draw a <category>" end to end
Isaac Sim + xArm7	Executes trajectories and reports measured pen-tip paths
External model services	Serve BioCLIP, FLUX, VLM judges (local or hosted)

Primary Use Cases

Use Case	Description
UC1: Build Dataset	Fetch, curate and normalize photos into subjects for chosen categories
UC2: Generate and Validate Drawings	Produce line drawings and accept/retry/reject them automatically
UC3: Export Gold Trajectories	Select the best drawing per angle bucket and export pen strokes
UC4: Draw in Simulation	Draw a requested category's gold trajectory with the xArm7
UC5: Measure Tracking Accuracy	Compute RMSE, max error and draw time for every run
UC6: Benchmark Models	Compare generators and judges on fixed evaluation sets
UC7: Reproduce and Audit a Run	Trace any result to its git SHA, config hash and inputs
UC8: Human Labeling (planned)	Collect accept/reject labels to calibrate the judges

5.1.2 Functional Requirements

The system must:

1. Fetch images and instance masks per category with versioned, idempotent reruns.
2. Reject duplicates, low-quality images, unusable masks and unsupported viewpoints.
3. Label each subject with species (BioCLIP) and viewing angle.
4. Generate a line drawing for each subject.
5. Validate drawings and retry generation with adjusted parameters up to a limit.
6. Select one drawing per category × subcategory × angle bucket.
7. Export gold trajectories (≤ 6 strokes, 16–128 points each) with lineage and QA scores.
8. Draw a requested category in simulation, or report that none is available.
9. Record tracking metrics and provenance for every robot run.
10. Benchmark candidate models with shared metrics and statistics.

5.1.3 Non-Functional Requirements

Performance Requirements

- Drawing generation: median ≤ 30 s per subject on a 16 GB laptop GPU (measured 24.2 s)
- Robot drawing: ≤ 600 s simulated time per drawing, the planned real-robot budget (longest 562.9 s)
- Simulation boot: Isaac Sim and its MCP extension ready within 3 minutes (measured ~155 s)

Scalability

- Stages are separate scripts with a category filter, ready to become Airflow tasks
- New categories and models are added through configuration, not code changes

Reliability

- Fail closed: a drawing that fails validation is never exported; a missing category returns `not_available`
- Idempotent reruns; long stages checkpoint every 5 samples; out-of-memory kills auto-resume in benchmarks

- Unsafe IK solutions (non-finite or out of joint limits) are rejected; timeouts end a run in a FAILED state

Resource Limits

- Only one large model resident at a time (FLUX needs ~20 GB host RAM; FLUX + Ollama together was OOM-killed)

Security

- API keys only in a gitignored .env file
- The Isaac Sim extension socket accepts arbitrary Python, so it binds to localhost only

Maintainability and Reproducibility

- Locked dependencies (uv.lock); configuration in YAML; 36 pipeline tests and 49 simulation tests
- Every robot run stamped with git SHA and config hash; the frozen baseline tagged baseline-v1 on GitHub

5.2 System Design

5.2.1 Multi-Stage Architecture

The end-to-end architecture is shown in Section 4.2.2. Each stage reads the previous stage's NDJSON manifest and writes its own, so any stage can be rerun, inspected or replaced on its own.

AI Components

- Viewpoint classifier (mask geometry)
- BioCLIP species labeler
- FLUX.1-schnell outline rectifier (alternatives: OmniGen2, FLUX.1-Kontext, SD1.5 + ControlNet, informative-drawings)
- OpenCV structural pre-screen
- VLM judge (qwen2.5vl:7b today; qwen3-vl-8b selected)
- LangGraph retry agent
- Lula IK controller with waypoint state machine
- Behavior cloning policy (prepared)

5.2.2 Platform Integration

- **WSL2 (Ubuntu 24.04)** hosts the code, data and pipeline.
- **Windows** runs Isaac Sim 6.0.1, because its GUI needs Vulkan, which WSL2 does not provide.
- Isaac Sim reads the project directly from WSL through \\wsl.localhost UNC paths, so nothing is copied.
- **MCP**: the isaacsim-mcp-server exposes 42 scene and simulation commands on localhost:8766. The wildtrace-sim server adds six workflow tools (sim_status, sim_start, setup_scene, draw, draw_result, sim_shutdown). Because WSL2 runs in NAT mode and

cannot reach Windows localhost, commands are relayed through a short-lived Windows Python process.

- **Ollama** serves local VLMs; **OpenRouter** serves hosted VLMs for benchmarks; the **Hugging Face Hub** supplies FLUX, OmniGen2 and BioCLIP weights.
- **GitHub** tracks issues (#10–#19) and the baseline-v1 release tag.

5.2.3 Data Management Architecture

Data Layers

Layer	Location	Contents	Volume
Bronze	outputs/bronze/	Raw images, masks, metadata; fetch ledger and curation manifests	540 samples, 5.6 GB
Silver	outputs/silver/	Normalized images, isolated subjects, crops, line diagrams, checkpoints	186 subjects, 353 MB
Gold	outputs/gold/	Selected diagrams, SVG, trajectories, gold_samples.ndjson	39 records, 5.4 MB
Simulation	isaac-sim/outputs/mcp_sessions/	job.json, executed paths, run_metrics.json, plots, captures	15 runs
Benchmarks	benchmarks/, outputs/benchmarks/	Evaluation sets, per-attempt logs, metrics, referee verdicts, reports	r1, r2
Reports	docs/benchmarks/, isaac-sim/outputs/reports/	Markdown and Word results, baseline report	—

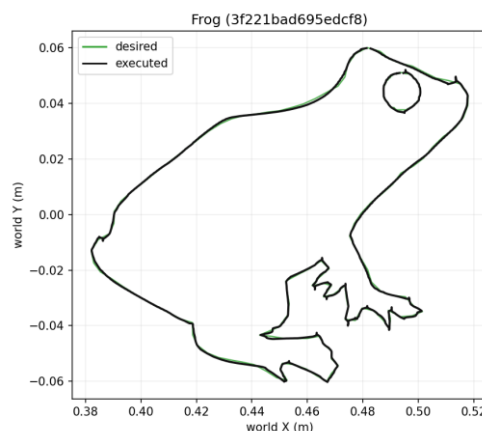
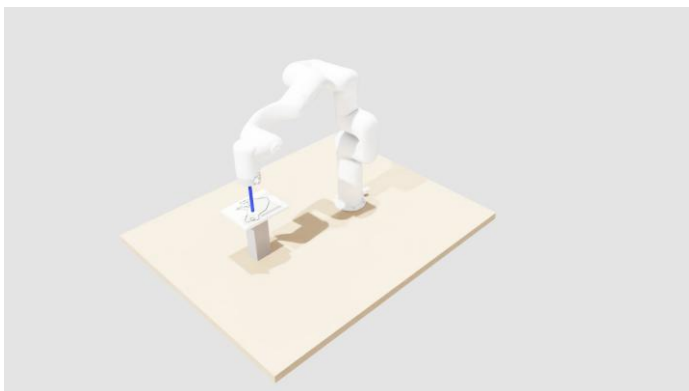
Gold Record Fields

- Identity: sample_id, category, subcategory, angle_bucket, task_type
- Trajectory: coordinate frame (normalized canvas), strokes with points and pen state, stroke_count, point_count
- QA scores: segmentation score, view confidence, diagram validation score
- Lineage: source image, mask path, diagram attempt, generator settings, validator model, timestamps

5.2.4 User Interface

Users interact through three surfaces:

- **Isaac Sim viewport:** shows the xArm7 drawing on the paper in real time, with desired and executed paths overlaid.
- **Agent chat over MCP:** an operator asks "draw a frog"; the agent provisions a trajectory, starts the simulation, waits for the run and reports RMSE, draw time and the comparison plot.
- **Notebook and reports:** pipeline/pipeline.ipynb runs and inspects every stage. Contact sheets and Word reports summarize benchmark results.



Isaac Sim Viewport after Drawing frog_3f221bad, and Desired vs Executed Path

5.3 Intelligent Solution

5.3.1 Integrated AI Framework

Model 1 is the curation stage: a geometric viewpoint classifier and BioCLIP. It reduced 540 Open Images samples to 186 usable subjects (34.4%) and labeled each with species (mean confidence 0.885) and viewing angle (mean confidence 0.674).

Model 2 is the FLUX.1-schnell outline rectifier, a 4-bit GGUF rectified-flow transformer conditioned on the subject's silhouette contour. It generated first drawings at a median 24.2 s per subject and, in benchmark r2, passed the 235B strong referee on 20 of 20 subjects.

Model 3 is the validation agent: an OpenCV pre-screen plus a VLM judge inside a LangGraph retry loop. It accepted 132 of 186 drawings (71.0%). Benchmark r2 showed the current 7B judge does not discriminate (κ 0.00 against the referee), so qwen3-vl-8b (κ 0.83) is selected to replace it.

Model 4 is trajectory extraction, which converted the best drawing per angle bucket into 39 gold trajectories (mean 4.46 strokes, 379.5 points).

Model 5 is the xArm7 drawing controller with Lula IK and a velocity-gated waypoint state machine. Under baseline-v1 it tracked 8 gold trajectories at a mean 0.330 mm RMSE, 4.3× better than the pre-baseline controller, finishing every drawing within 10 minutes.

Model 6 is the behavior cloning policy, an MLP that predicts bounded Cartesian deltas. It is implemented and tested and will be trained once multiple demonstrations are recorded from the gold set.

5.3.2 Composite Validation Score and Routing

Combined score: $S = 0.5 \cdot \text{OpenCV score} + 0.5 \cdot \text{VLM score}$, used to rank drawings within each angle bucket.

Condition	Action
OpenCV pass, VLM verdict true, VLM score ≥ 0.55	Accept
Any check fails, attempts < max	Adjust strength (and VLM-suggested params); regenerate
Any check fails, attempts = max (5 in production, 3 in r2)	Reject with failure flags
Several accepted drawings in one angle bucket	Keep the highest S as gold

This turns validation into routing rather than a single pass/fail, and every decision is logged with its scores.

5.3.3 Failure Handling

What happens if the system faces any of the following situations:

- Drawing fails the structural check → the VLM call is skipped and the agent retries with new parameters
- Generator returns a blank image → seeds vary per attempt, so a blank seed is not repeated
- Out of memory → the generator unloads after each call; benchmark runs auto-resume after an OOM kill
- Requested category has no gold trajectory → not_available is reported and nothing is written
- IK solution non-finite or outside joint limits → rejected; repeated failures or timeouts end the run as FAILED
- Isaac Sim interrupted during boot → it is launched as a detached Windows process (WMI), so a closing WSL shell cannot interrupt it
- Simulation play command lost → the draw tool confirms the timeline state after stop and play, and retries once

The robot only ever draws trajectories that passed validation.

5.3.4 End-to-End Output Quality Validation

Input

Operator request: "draw a frog".

Processing

1. provision_and_draw finds 5 frog gold trajectories and picks an undrawn one (3f221bad; 2 strokes, 224 points).
2. The wildtrace-sim server launches Isaac Sim, builds the scene (physics, robot, paper, camera, controller graph) and starts the run.
3. The controller maps the drawing to the paper, solves IK each tick and draws both strokes.
4. run_metrics.json, a desired-vs-executed plot and a viewport capture are written.

Output

- Status finished, all states visited, 533 pen-down points
- Tracking RMSE 0.366 mm, max error 1.024 mm, draw time 277.1 s
- Config hash 15e6428dffca matches baseline-v1, so the run is directly comparable

Empirical Validation

The new run landed inside the baseline-v1 range (0.270–0.394 mm). The plot shows small hook-shaped overshoots at sharp corners (eye, toes), the remaining "knots" issue tracked as #11.

5.4 System Support Environment

The system runs on a single GPU laptop with a split Linux/Windows setup, designed for experimentation, reproducibility and a later move to a real robot.

5.4.1 Technology Stack

1. Core Development Environment

- Python 3.12 with uv-locked dependencies
- PyTorch 2.11 (CUDA 13.0)
- Hugging Face diffusers, transformers, accelerate, gguf
- OpenCV, NumPy, Pillow, open-clip

2. Models and Inference

- FLUX.1-schnell GGUF Q4_K_S (local GPU, CPU offload)
- BioCLIP (local GPU)
- Ollama (qwen2.5vl:7b) and OpenRouter (qwen3-vl-8b, qwen3-vl-235b) VLM judges
- ONNX Runtime for lightweight models

3. Agents and Orchestration

- LangGraph state graph for validation and retry
- Stage scripts sharing one runner, plus an end-to-end notebook
- MCP servers: isaacsim-mcp-server (42 commands) and wildtrace-sim (6 workflow tools)

4. Simulation

- NVIDIA Isaac Sim 6.0.1, PhysX, Lula kinematics
- xArm7 USD generated from the official URDF
- Action Graph ScriptNode controller ticking at 60 Hz

5. Evaluation and Reporting

- eval_metrics.py (8 metric groups), bootstrap CIs, sign tests, Cohen's κ
- Markdown and Word reports; matplotlib figures and contact sheets

6. Version Control and Governance

- Git and GitHub (issues, baseline-v1 tag)
- Config hash + git SHA stamped into every robot run

- pytest suites for the pipeline (36) and simulation (49)

5.4.2 Infrastructure Capabilities

- GPU-accelerated generation, labeling and judging on a 16 GB laptop GPU
- Local or hosted VLM judges, selected by configuration
- Idempotent, checkpointed, category-filtered pipeline stages
- Physics simulation of a 7-DOF arm with real-time visualization
- Agent-driven, one-command simulation runs over MCP
- Memory-safe execution (one large model resident, OOM auto-resume)
- Reproducible benchmarks with fixed seeds and frozen evaluation sets
- Full audit trail from photo to robot metrics

5.4.3 Machine Learning Data Flow Architecture

1. Open Images images and masks are fetched into bronze storage with a versioned ledger.
2. Curation filters samples and labels viewpoint and species.
3. Silver normalization isolates and crops each subject.
4. FLUX.1-schnell generates a line drawing from the subject's silhouette.
5. The OpenCV pre-screen and VLM judge validate it; the LangGraph agent retries failures.
6. The best drawing per angle bucket is selected.
7. Pen trajectories are extracted and exported as gold records.
8. provision_and_draw queues a gold trajectory for Isaac Sim.
9. The xArm7 controller draws it and records tracking metrics with provenance.
10. Benchmarks and reports compare models and controller versions.

5.4.4 Reproducibility and Auditability

The system ensures:

- Every robot run records the git SHA and a hash of its configuration
- The frozen controller configuration is tagged baseline-v1 (commit 74e32f9, config hash 15e6428dffca)
- Every pipeline record carries lineage back to its source image, mask and generator settings
- Benchmarks use seeded evaluation sets and log every attempt, judge call, latency and cost
- Dependencies are locked with uv

This environment ensures the system is:

- Reproducible
- Traceable
- Modular (models swap through configuration)
- Resource-aware
- Ready for real-robot integration

5.5 Limitations and Future Work

- **Gravity (#13):** the robot currently runs with gravity disabled. Normal gravity needs a gravity-compensation controller before the effort limits can hold the home pose.
- **Corner knots (#11):** small overshoots remain at sharp corners.
- **Human labels:** the 60-drawing label set has not been labeled yet, so judge accuracy is measured against a strong-model proxy.
- **Untested candidates:** FLUX.1-Kontext, SD1.5 + ControlNet, informative-drawings, qwen3.5 and qwen3-vl-30b-a3b are implemented or planned but not yet benchmarked.
- **Pose fidelity:** FLUX draws stock poses ($\text{IoU} \approx 0.2$); stronger contour conditioning is the next experiment.
- **Coverage gaps:** Butterfly has no masks; Fish and Horse lack some angle buckets.
- **Robustness (#19)** to paper height, tilt, noise and delay is not yet tested.
- **Behavior cloning** needs more demonstrations before training.

References

- [1] A. Kuznetsova et al., "The Open Images Dataset V4: Unified image classification, object detection, and visual relationship detection at scale," International Journal of Computer Vision, 2020.
- [2] R. Benenson, S. Popov, and V. Ferrari, "Large-scale interactive object segmentation with human annotators," in Proc. CVPR, 2019.
- [3] S. Stevens et al., "BioCLIP: A Vision Foundation Model for the Tree of Life," in Proc. CVPR, 2024.
- [4] A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision," in Proc. ICML, 2021.
- [5] Black Forest Labs, "FLUX.1 [schnell]," Model Card, Hugging Face, 2024. [Online]. <https://huggingface.co/black-forest-labs/FLUX.1-schnell>
- [6] city96, "FLUX.1-schnell-gguf," GGUF quantizations, Hugging Face, 2024. [Online]. <https://huggingface.co/city96/FLUX.1-schnell-gguf>
- [7] Black Forest Labs et al., "FLUX.1 Kontext: Flow Matching for In-Context Image Generation and Editing in Latent Space," arXiv:2506.15742, 2025.
- [8] C. Wu et al., "OmniGen2: Exploration to Advanced Multimodal Generation," arXiv:2506.18871, 2025.
- [9] S. Bai et al., "Qwen2.5-VL Technical Report," arXiv:2502.13923, 2025.
- [10] Qwen Team, "Qwen3-VL" (8B-Instruct and 235B-A22B-Instruct), Model Cards, Hugging Face, 2025.
- [11] R. Rombach et al., "High-Resolution Image Synthesis with Latent Diffusion Models," in Proc. CVPR, 2022.
- [12] L. Zhang, A. Rao, and M. Agrawala, "Adding Conditional Control to Text-to-Image Diffusion Models," in Proc. ICCV, 2023.
- [13] C. Chan, F. Durand, and P. Isola, "Learning to generate line drawings that convey geometry and semantics," in Proc. CVPR, 2022.
- [14] LangChain, "LangGraph," Documentation, 2024. [Online]. <https://langchain-ai.github.io/langgraph/>
- [15] NVIDIA, "Isaac Sim Documentation" (Lula kinematics solver, motion generation), 2026. [Online]. <https://docs.isaacsim.omniverse.nvidia.com/>

- [16] UFACTORY, "xarm_ros2," GitHub Repository. [Online]. https://github.com/xArm-Developer/xarm_ros2
- [17] Anthropic, "Model Context Protocol Specification," 2024. [Online]. <https://modelcontextprotocol.io/>
- [18] Ollama, "Ollama," GitHub Repository. [Online]. <https://github.com/ollama/ollama>
- [19] OpenRouter, "OpenRouter API Documentation." [Online]. <https://openrouter.ai/docs>
- [20] J. Cohen, "A coefficient of agreement for nominal scales," Educational and Psychological Measurement, vol. 20, no. 1, 1960.
- [21] B. Efron, "Bootstrap methods: another look at the jackknife," The Annals of Statistics, vol. 7, no. 1, 1979.
- [22] F. Perazzi et al., "A Benchmark Dataset and Evaluation Methodology for Video Object Segmentation," in Proc. CVPR, 2016.
- [23] D. A. Pomerleau, "ALVINN: An Autonomous Land Vehicle in a Neural Network," in Advances in Neural Information Processing Systems, 1989.
- [24] "Rapid Learning of Dexterous In-Hand Pen Writing," arXiv:2609.11775, 2026 (as cited in the WildTrace baseline-v1 report).
- [25] "Six-DOF hybrid robotic arm trajectory tracking," arXiv:2407.19826, 2024 (as cited in the WildTrace baseline-v1 report).